

AUFGABE ZUR VORLESUNG

Paketbilder – Upload mit Transactional Outbox

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema Bild-Upload, File-Service, SQLite-Outbox, Wiederholungen
Dozent Dr. Ulrich Winkler
Semester 5. Semester · Informationstechnik

Voraussetzung: Übungen 01 und 02. Sie haben einen Parcel-Service mit `parcels/<id>.json` und `tracking/<id>.log` unter `$DATA_DIR`. **Diese Übung ist unbewertet.**

Ziel: Kunden laden ein echtes Bild zu einem Paket hoch. Der Parcel-Service nimmt das Bild entgegen und legt es zunächst als **temporäre Datei** ab. Ein eigener **File-Service** sortiert es später in den endgültigen Ordner des Pakets ein. Beide Dienste und das Relay schreiben Sie in Python 3.12 als Erweiterung Ihres bestehenden Projekts unter `loesung/`.

Was Sie neu implementieren

- **Bild-Upload im Parcel-Service:** `POST /parcels/{parcel_id}/images` empfängt eine PNG-Datei und eine vom Client gewählte `image_id` (z. B. `foto-1`) als `multipart/form-data`. Speichern Sie die empfangenen Bytes zuerst in einer Datei unter `$STAGING_DIR`. Antworten Sie erst, wenn die temporäre Datei und der zugehörige Datenbankauftrag dauerhaft gespeichert sind. Das Bild hat zunächst den Zustand `PENDING`.
- **SQLite-Outbox:** Speichern Sie Bild-Metadaten (`parcel_id`, `image_id`, Dateipfad, Prüfsumme, Status) und einen Auftrag `StoreImageRequested` in **einer SQLite-Transaktion** unter `$DATA_DIR`. Der Auftrag enthält den Pfad der temporären Datei und die erwartete Prüfsumme. Die Paket- und Tracking-Dateien aus 01/02 bleiben bestehen.
- **File-Service:** Ein eigener Prozess erhält vom Relay den Auftrag und prüft die Prüfsumme. Dann sortiert er die temporäre Datei unter `$FILE_DATA_DIR/files/P-4711/foto-1.png` ein. Für diese Übung laufen beide Dienste auf demselben Rechner; `$STAGING_DIR` ist für beide sichtbar und liegt auf demselben Dateisystem wie `$FILE_DATA_DIR`. Der File-Service meldet Erfolg erst, wenn die Datei am Ziel sicher gespeichert ist.
- **Relay:** Ein separates Python-Programm liest regelmäßig offene Outbox-Aufträge und beauftragt den File-Service per HTTP. Der Relay sendet **Pfad und Prüfsumme**, nicht die Bildbytes. Nach bestätigtem Erfolg setzt er in **einer SQLite-Transaktion** den Bildstatus auf `READY` und markiert den Auftrag als erledigt. Bei einem Fehler bleibt der Auftrag offen und wird beim nächsten Durchlauf erneut versucht. Halten Sie während des HTTP-Aufrufs keine SQLite-Transaktion offen.

Ein SQLite-Commit und das Verschieben der Datei sind **nicht gemeinsam atomar**. Deshalb muss der File-Service bei einer Wiederholung erkennen, dass das Bild bereits am Ziel liegt: Passt dort die Prüfsumme, bestätigt er den Auftrag erneut, auch wenn die temporäre Datei inzwischen fehlt. Ein anderer Inhalt darf das vorhandene Bild nicht überschreiben. Ein doppelter Upload mit derselben `image_id` und demselben Inhalt darf keinen weiteren Outbox-Auftrag erzeugen; bei anderem Inhalt melden Sie einen Konflikt.

Prüfen Sie drei Fälle: (1) File-Service ist zunächst aus und wird später gestartet; (2) der Relay stürzt nach dem Verschieben, aber vor seinem SQLite-Commit ab; (3) der Parcel-Service stürzt nach dem Schreiben der temporären Datei, aber vor dem SQLite-Commit ab. In Fall 3 kann eine unreferenzierte temporäre Datei übrig bleiben: Wie finden und entfernen Sie solche Dateien, ohne noch offene Aufträge zu beschädigen?